

Nama : Moch Riezky Dwi Kuswanto

NRP : 152024133

Guided Scheduling adalah algoritma penjadwalan adaptif di mana tugas diberikan dalam potongan (*chunk*) yang ukurannya menurun secara bertahap.

Tujuan: Mengurangi beban komunikasi (*low overhead*) di awal dengan mengirim potongan besar, namun tetap menjaga keseimbangan di akhir agar semua prosesor selesai bersamaan (*good balance*).

Rumus Utama: $\text{Chunk} = \max(\text{pRemainingWork}, \text{Min Chunk})$

ALGORITMA Guided_Scheduling:

INPUT: total_tasks (W), num_processors (p)

SET remaining_work = W

SET min_chunk = 1

FOR tiap Processor (Paralel):

WHILE remaining_work > 0:

LOCK (mengunci sisa_tugas agar tidak rebutan)

hitung chunk = MAX(remaining_work / p, min_chunk)

update remaining_work = remaining_work - chunk

UNLOCK

EKSEKUSI chunk tersebut

END WHILE

END FOR

```
import threading

import math

import time

import random


# Inisialisasi sesuai variabel di Slide

W = 100      # Total Work (Slide 181)

p = 4        # Number of Processors (Slide 181)

min_chunk = 1    # Minimum Chunk Size

remaining_work = W

lock = threading.Lock()


def worker(processor_id):

    global remaining_work

    tasks_done = 0


    while True:

        with lock:

            if remaining_work <= 0:

                break
```

```

# Rumus Guided Scheduling dari Slide 21

# Chunk = max(Remaining Work / p, Min Chunk)

chunk_size = max(math.ceil(remaining_work / p), min_chunk)

remaining_work -= chunk_size

current_batch = chunk_size


# Simulasi pengerjaan tugas

tasks_done += current_batch

print(f"[Processor P {processor_id}] Mengambil {current_batch} tugas.
Sisa antrian: {remaining_work}")


# Simulasi durasi pengerjaan (beban kerja bervariasi)

time.sleep(random.uniform(0.1, 0.3))


print(f"--- P {processor_id} Selesai. Total dikerjakan: {tasks_done} tugas
---")


# Menjalankan p threads (Processor)

threads = []

print(f"Memulai Load Balancing dengan {p} Processor untuk {W}
Tugas...\n")

for i in range(p):

    t = threading.Thread(target=worker, args=(i,))

    threads.append(t)

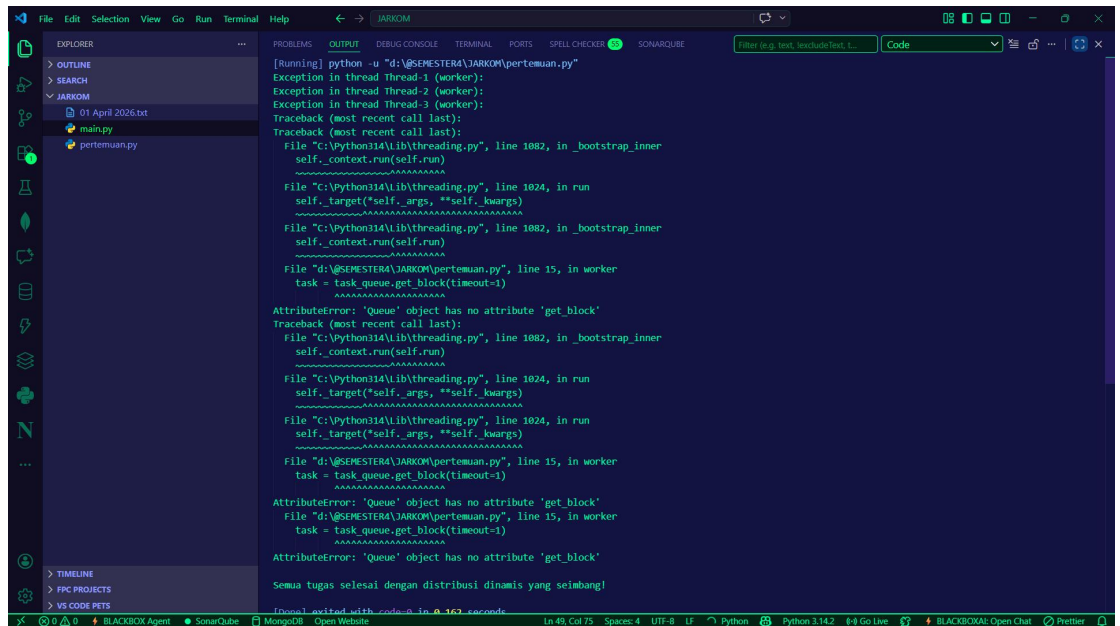
    t.start()

```

for t in threads:

t.join()

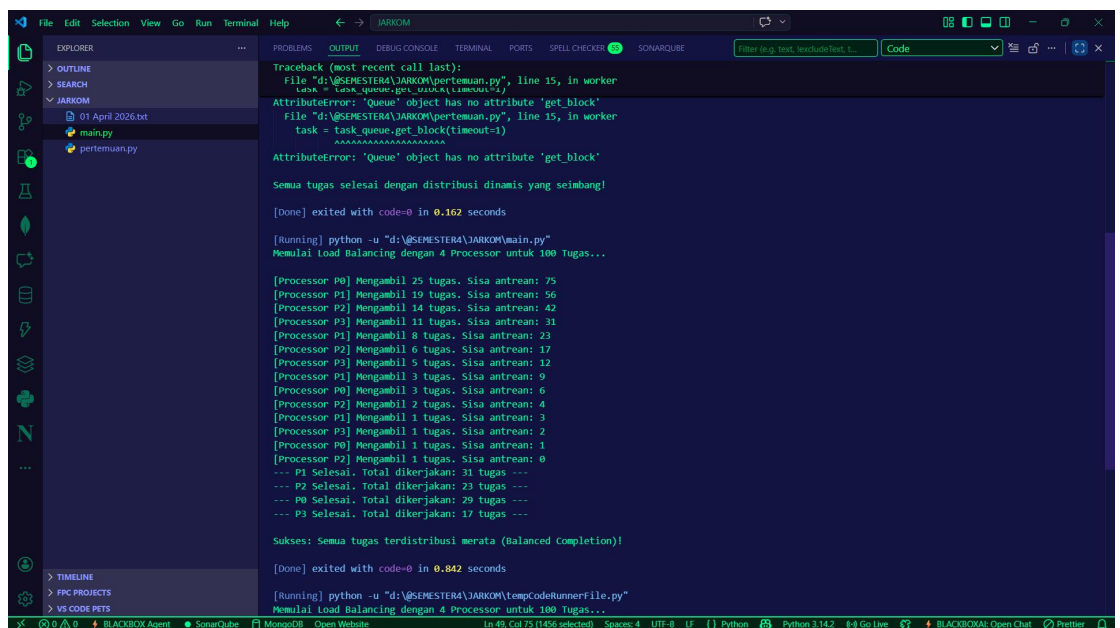
print("\nSukses: Semua tugas terdistribusi merata (Balanced Completion)!")



The screenshot shows a VS Code window with a terminal running a Python script. The script is located at 'd:\@SEMESTER4\JARKOM\pertemuan.py'. The terminal output shows the following error:

```
[Running] python -u "d:\@SEMESTER4\JARKOM\pertemuan.py"
Exception in thread Thread-1 (worker):
Exception in thread Thread-2 (worker):
Exception in thread Thread-3 (worker):
Traceback (most recent call last):
Traceback (most recent call last):
File "C:\Python314\lib\threading.py", line 1082, in _bootstrap_inner
self._context.run(self._run)
File "C:\Python314\lib\threading.py", line 1024, in run
self._target(*self._args, **self._kwargs)
File "C:\Python314\lib\threading.py", line 1082, in _bootstrap_inner
self._context.run(self._run)
File "d:\@SEMESTER4\JARKOM\pertemuan.py", line 15, in worker
task = task_queue.get_block(timeout=1)
AttributeError: 'Queue' object has no attribute 'get_block'
Traceback (most recent call last):
File "C:\Python314\lib\threading.py", line 1082, in _bootstrap_inner
self._context.run(self._run)
File "C:\Python314\lib\threading.py", line 1024, in run
self._target(*self._args, **self._kwargs)
File "C:\Python314\lib\threading.py", line 1024, in run
self._target(*self._args, **self._kwargs)
File "d:\@SEMESTER4\JARKOM\pertemuan.py", line 15, in worker
task = task_queue.get_block(timeout=1)
AttributeError: 'Queue' object has no attribute 'get_block'
File "d:\@SEMESTER4\JARKOM\pertemuan.py", line 15, in worker
task = task_queue.get_block(timeout=1)
AttributeError: 'Queue' object has no attribute 'get_block'
File "d:\@SEMESTER4\JARKOM\pertemuan.py", line 15, in worker
task = task_queue.get_block(timeout=1)
AttributeError: 'Queue' object has no attribute 'get_block'

Semua tugas selesai dengan distribusi dinamis yang seimbang!
[Done] exited with code=0 in 0.162 seconds
```



The screenshot shows a VS Code window with a terminal running a Python script. The script is located at 'd:\@SEMESTER4\JARKOM\main.py'. The terminal output shows the following successful execution:

```
[Running] python -u "d:\@SEMESTER4\JARKOM\main.py"
Memulai Load Balancing dengan 4 Processor untuk 100 Tugas...

[Processor P0] Mengambil 25 tugas. Sisa antrian: 75
[Processor P1] Mengambil 19 tugas. Sisa antrian: 56
[Processor P2] Mengambil 14 tugas. Sisa antrian: 42
[Processor P3] Mengambil 11 tugas. Sisa antrian: 31
[Processor P0] Mengambil 8 tugas. Sisa antrian: 23
[Processor P2] Mengambil 6 tugas. Sisa antrian: 17
[Processor P3] Mengambil 5 tugas. Sisa antrian: 12
[Processor P1] Mengambil 3 tugas. Sisa antrian: 9
[Processor P0] Mengambil 3 tugas. Sisa antrian: 6
[Processor P2] Mengambil 2 tugas. Sisa antrian: 4
[Processor P1] Mengambil 1 tugas. Sisa antrian: 3
[Processor P3] Mengambil 1 tugas. Sisa antrian: 2
[Processor P0] Mengambil 1 tugas. Sisa antrian: 1
[Processor P2] Mengambil 1 tugas. Sisa antrian: 0
--- P1 Selesai. Total dikerjakan: 31 tugas ---
--- P2 Selesai. Total dikerjakan: 23 tugas ---
--- P0 Selesai. Total dikerjakan: 29 tugas ---
--- P3 Selesai. Total dikerjakan: 17 tugas ---

Sukses: Semua tugas terdistribusi merata (Balanced Completion)!
[Done] exited with code=0 in 0.842 seconds

[Running] python -u "d:\@SEMESTER4\JARKOM\tempCodeRunnerFile.py"
Memulai Load Balancing dengan 4 Processor untuk 100 Tugas...
```

Analisis Hasil

0 pertama kali mengambil **25 tugas** ($100/4$).

Setelah itu, **P1** melihat sisa 75, maka ia mengambil **19 tugas** ($75/4=18.75$, dibulatkan ke atas).

Mekanisme ini memastikan di awal tidak banyak terjadi "antrean" (sinkronisasi), tapi di akhir tidak ada prosesor yang menganggur karena potongan tugasnya mengecil menjadi **1 tugas per ambil**.